

Documentation and Code Conventions

Created by <Academia de Código_>

Javadoc

Javadoc is a tool for generating API documentation in HTML format from comments in source code, keeping the documentation in sync with the software.

Class Summary	
AdditionalMatchers	See Matchers for general info about matchers.
ArgumentCaptor<T>	Use it to capture argument values for further assertions.
ArgumentMatcher<T>	Allows creating customized argument matchers.
BDDMockito	Behavior Driven Development style of writing tests uses <code>//given</code> <code>//when</code> <code>//then</code> comments as fundamental parts of your test methods.
BDDMockito.BDDOngoingStubbingImpl<T>	
BDDMockito.BDDStubberImpl	
Matchers	Allow flexible verification or stubbing.
Mockito	
MockitoAnnotations	<code>MockitoAnnotations.initMocks(this);</code> initializes fields annotated with Mockito annotations.

Comments

Javadoc comments begin with `/**` and end with `*/`. They can appear immediately before a class, field (public class or instance variable), constructor or method definition

```
/** Class Description of MyClass */
public class MyClass {

    /** Field Description of myIntField */
    public int myIntField;

    /** Constructor Description of MyClass() */
    public MyClass() {

        // Do something ...

    }
}
```

Tags

The Javadoc tool parses special tags when they are embedded within a Java doc comment. These tags enable you to autogenerate a complete, well-formatted API from your source code.

```
// Description of MyClass
public class MyClass {
    // Description of myIntField
    public int myIntField;

    // Description of MyClass()
    // @throws MyException          Description of myException
    public MyClass() throws MyException {
        // Blah Blah Blah...
    }

    // Description of myMethod(int a, String b)
    // @param a                      Description of a
    // @param b                      Description of b
    // @return                      Description of c
    public Object myMethod(int a, String b) {
        Object c = null;
        // Blah Blah Blah...
        return c;
    }
}
```

Code Conventions

Set of guidelines for a specific programming language that recommends programming style, practices and methods for each aspect of a program written in this language.

- file organization
- indentation
- comments
- declarations
- statements
- white space
- naming conventions
- etc.

Why Have Code Conventions?

- 80% of the lifetime cost of a piece of software goes to maintenance
- Hardly any software is maintained for its whole life by the original author
- Code conventions improve the readability of the software, allowing engineers to understand new code more quickly and thoroughly.
- If you ship your source code as a product, you need to make sure it is as well packaged and clean as any other product you create

Java Code Conventions, Sun, 1997

Java Code Conventions

- Sun Java Code Conventions, dated from 1997 but still relevant
- Google Java Style Guide

Every major open-source project has its own style guide: a set of conventions (sometimes arbitrary) about how to write code for that project.

It is much easier to understand a large codebase when all the code in it is in a consistent style.

Google Style Guides

Any fool can write code that a computer can understand.
Good programmers write code that humans can understand.

Martin Fowler, Refactoring: Improving the Design of Existing Code